ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ 11

Օբյեկտի փոխանցումը ֆունկցիային

Աշխատանքի նպատակը: Տարբերակել օբյեկտի փոխանցումը արժեքով, հղումով և հաստատուն հղումով:

Տեսական մաս: Արժեքով փոխանցումը ստեղծում է պարամետրի պատճեն: Հղումով փոխանցումը թույլ է տալիս փոխել սկզբնական օբյեկտը: const հղումը կիրառվում է առանց պատճենման ընթերցելու համար: Արտաքին սովորական ֆունկցիան private տվյալները ստանում է public մեթոդների միջոցով:

Աշխատանքի կատարման կարգը

1. Կարդալ ընդհանուր պահանջները և նշանակված անհատական տարբերակը:

2. Սահմանել դասի դաշտերը, դրանց սահմանափակումները և մեթոդները:

3. Իրականացնել պահանջվող գործողությունները և ստուգել սխալ մուտքերը:

4. Համեմատել ստուգիչ օրինակի արդյունքը ձեռքով ստացված հաշվարկի հետ:

Աշխատակիցների ժամային վճարներ

Տվյալներ: Ստեղծել PayrollRegister դաս՝ պահելով աշխատակիցների ամսվա աշխատաժամերի զանգվածը և մեկ ժամի p դրամ վճարը:

Հաշվարկներ: Հաշվել աշխատավարձերի ընդհանուր գումարը՝ Σ(hours·p), գտնել ամենաբարձր աշխատավարձ ունեցող աշխատակցի ինդեքսը և 160 ժամից ավելի աշխատած անձանց քանակը:

Թույլատրելի արժեքներ: ժամերը ոչ բացասական իրական թվեր են, p>0:

Ֆունկցիաներ: analyze(const PayrollRegister&) արտածում է երեք հաշվարկները: changeCopy(PayrollRegister object, int index, double

value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const PayrollRegister&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: hours={120,168,150,180}, p=1500: Ինդեքսները սկսվում են 0-ից:

```
#include <iostream>
#include <vector>
#include <numeric>
#include <algorithm>
#include <iomanip>
class PayrollRegister {
private:
    std::vector<double> hours;
    double p; (p > 0)
public:
    PayrollRegister() : p(1500) {
}
    PayrollRegister(const std::vector<double>& h, double wage) {
        if (wage <= 0) {
            throw std::invalid_argument; }
        for (double val : h) {
            if (val < 0) {
                throw std::invalid_argument; } }
        hours = h;
        p = wage; }
    size_t size() const { return hours.size(); }
    double getHour(size_t index) const {
        if (index >= hours.size()) {
            throw std::out_of_range; }
        return hours[index]; }
    double getP() const { return p; }
    bool set(size_t index, double value) {
        if (index >= hours.size() || value < 0) {
            return false;
        }
        hours[index] = value;
        return true; }
    void print() const {
        std::cout << "PayrollRegister [Hours: {";
        for (size_t i = 0; i < hours.size(); ++i) {
            std::cout << hours[i] << (i + 1 < hours.size() ? ", " : "");
        }
        std::cout << "}, p: " << p << "]" << std::endl; }
    sum(hours * p)
    double calculateTotalPayroll() const {
        double total = 0;
        for (double h : hours) {
            total += h * p; }
        return total; }
    int findHighestSalaryIndex() const {
        if (hours.empty()) return -1;
        size_t bestIdx = 0;
        double maxSalary = hours[0] * p;
        for (size_t i = 1; i < hours.size(); ++i) {
            double sal = hours[i] * p;
            if (sal > maxSalary) {
                maxSalary = sal;
```

```
            bestIdx = i; } }
    return static_cast<int>(bestIdx); }
int countOver160Hours() const {
    int count = 0;
    for (double h : hours) {
        if (h > 160) {
            count++; } }
    return count; }
void reverse() {
    std::reverse(hours.begin(), hours.end()); } };
return 0;}
```